Trabalho de Conclusão de Curso de Engenharia da Computação

Protótipo para detecção, localização e seleção automática de ação por sensoriamento

Modelo de Banco de Dados

Gabriel Vasconcellos de Morais – Matheus Emanoel Gomes Souza

Orientador: Anker Douglas Loss

Curso: Engenharia da Computação — Faculdade do Centro Leste (UCL)

1 Apresentação do Modelo de Dados

O RadarTorres é um protótipo que detecta alvos por sensoriamento, seleciona automaticamente a torre mais adequada e permite o acionamento manual ou automático de um indicador demonstrativo. Para cumprir esse papel, o sistema precisa preservar, entre execuções, dados de naturezas distintas: contas de usuário e seus perfis de acesso, o histórico de objetos detectados pelo radar, a auditoria de toda ação de acionamento e de toda troca de modo de operação, e as preferências de interface de cada usuário.

Por não depender de um servidor de banco de dados, o protótipo prioriza simplicidade de instalação e execução local/offline — coerente com o estágio atual do projeto (protótipo acadêmico, uso demonstrativo). As seções seguintes descrevem a tecnologia de persistência empregada, por que essa escolha foi feita, as especificações técnicas do SGBD e as entidades e relacionamentos do modelo atual.

2 Tecnologia e Implementação

O protótipo utiliza **SQLite** como único mecanismo de persistência. O contêiner de injeção de dependência (`App.xaml.cs:141--146`) registra uma implementação `Sqlite*Repository` para cada uma das 5 tabelas do sistema (`Usuario`, `ObjetoDetectado`, `AcaoRealizada`, `ModoAtualTorre`, `PreferenciasUsuario`). Não existem mais classes de persistência em CSV no repositório — a migração CSV → SQLite, antes descrita como plano futuro, já foi concluída.

O arquivo de dados reside em `%AppData%\RadarTorres\Data\radartorres.db` (pasta do usuário do Windows, sem exigir privilégio de administrador). Falhas de escrita são enfileiradas em `pending-writes.sql` (`PendingWriteQueue.cs`) e reaplicadas automaticamente na próxima inicialização, evitando perda silenciosa de dados.

2.1 Especificações técnicas do SGBD

2.2 Por que SQLite: argumento de segurança e leveza

A escolha do SQLite para o RadarTorres não é um atalho de conveniência — decorre diretamente do perfil de execução do sistema: um aplicativo desktop único, instalado na máquina do operador, sem rede corporativa, sem múltiplos servidores concorrentes e com volume de dados compatível com um protótipo demonstrativo. Dentro desse perfil, SQLite oferece a combinação de **segurança** e **leveza** que um SGBD cliente-servidor (PostgreSQL, SQL Server, MySQL) não justificaria:

- **Superfície de ataque menor:** por não expor porta de rede nem processo de servidor separado, o SQLite elimina uma classe inteira de riscos (varredura de porta, autenticação de rede, man-in-the-middle entre aplicação e banco) — o banco só é acessível por quem já tem acesso ao arquivo local, no próprio disco do usuário.

Tabela 1: Especificações técnicas da persistência

Item	Valor
SGBD	SQLite (embarcado, arquivo único, sem servidor)
Driver/linguagem de acesso	`Microsoft.Data.Sqlite` versão **9.0.0**, consumido em C#/.NET 9
Versão do engine	Embutida no pacote via `SQLitePCLRaw`; não pinada separadamente pelo projeto
Codificação	UTF-8 (padrão SQLite para colunas `TEXT`)
Collation	`BINARY` (padrão; nenhuma coluna usa `COLLATE` customizado)
Integridade referencial	`PRAGMA foreign_keys = ON`
Modelo de concorrência	Single-writer / multi-reader (padrão SQLite, arquivo local)

- **Integridade garantida mesmo sendo leve:** `PRAGMA foreign_keys = ON` aplica chaves estrangeiras reais, e o mecanismo ACID nativo do SQLite (transações atômicas por arquivo) evita corrupção parcial em caso de queda de energia ou encerramento abrupto — sem exigir o processo e a configuração de um SGBD completo para obter essa garantia.
- **Senhas nunca em texto puro:** a camada de aplicação usa PBKDF2-HMACSHA256 (100.000 iterações, salt de 128 bits por usuário), independente do SGBD escolhido — mas a ausência de um servidor de rede reduz também a superfície onde esse hash poderia ser interceptado em trânsito.
- **Zero instalação, zero administração:** não há usuário de banco, senha de administrador de SGBD, porta a abrir no firewall ou serviço do Windows a manter — o único artefato é um arquivo `.db` na pasta do próprio usuário, reduzindo pontos de falha e de configuração incorreta (más práticas de hardening de servidor não se aplicam a um banco que não existe como servidor).
- **Leveza compatível com o hardware do protótipo:** o motor SQLite é embutido no processo da aplicação (`Microsoft.Data.Sqlite`), sem overhead de IPC/rede e sem consumo de memória de um processo de SGBD separado — adequado ao uso em uma estação de operação única, sem exigir infraestrutura dedicada de banco de dados.
- **Resiliência a falhas sem complexidade adicional:** a fila de escritas pendentes (`PendingWriteQueue.cs`) cobre o único cenário de falha relevante para um arquivo local (banco temporariamente indisponível/bloqueado), sem precisar de um cluster ou réplica para alta disponibilidade — desnecessário na topologia de nó único do RadarTorres.

Em resumo: para o escopo atual do RadarTorres — uma aplicação local, de usuário único por instalação, sem exposição a rede — o SQLite entrega as garantias de integridade e auditoria que o sistema exige, com a menor superfície de ataque e o menor custo operacional possíveis. Caso o projeto evolua para múltiplos usuários concorrentes em rede, essa decisão seria revisitada em favor de um SGBD cliente-servidor.

3 Principais Entidades do Sistema

4 Modelo e Relacionamentos

A Figura 1 reproduz o Diagrama Entidade-Relacionamento do schema atual (arquivo `Modelo_Banco_de_Dados.md` e `schema.sql`, nesta mesma pasta).

`usuarios` se relaciona 1:1 com `preferencias_usuario` (chave `UsuarioId`, igual a `usuarios.Id`) e 1:N com `modo_atual_torre` — agora por chave estrangeira real (`ID_USER` → `usuarios.Id`), com

Tabela 2: Tabelas do schema atual (SQLite)

Tabela	Finalidade
usuarios	Contas de acesso, perfil (Administrador/Operador/Visualizador) e autenticação
objetos_detectados	Histórico de ocorrências de detecção (radar e câmera, por quadrante)
ocorrencia_estados	Auditoria das transições de estado de cada ocorrência (só inserção)
cameras	Câmera responsável por cada quadrante (Q1–Q4)
acoes_realizadas	Auditoria de acionamentos efetivamente executados (só inserção)
modo_atual_torre	Auditoria de troca de modo de operação (só inserção)
preferencias_usuario	Preferências de interface por usuário (tema, idioma, sidebar)

`PRAGMA foreign_keys = ON` garantindo a integridade. `objetos_detectados` se relaciona 1:N com `acoes_realizadas` (quando há alvo associado ao acionamento) e com `ocorrencia_estados` (histórico de transição); `cameras` se relaciona 1:N com `objetos_detectados` pelo quadrante monitorado. `objetos_detectados` não possui relacionamento persistido com `usuarios` — o campo `Dispositivo` identifica a fonte da detecção (ex.: "Arduino", "Simulador"), não um usuário.

5 Índices

Além das chaves primárias e das restrições `UNIQUE` (`usuarios.Login`, `preferencias_usuario.UsuarioId`), o schema define os índices da Tabela 3 para as consultas mais frequentes da aplicação (ordenação por data/hora, filtragem por torre e por usuário).

Tabela 3: Índices definidos em `schema.sql`

Índice	Tabela / coluna(s)	Tipo
idx_acoes_realizadas_torre	acoes_realizadas (ID_TORRE)	normal
idx_objetos_detectados_datahora	objetos_detectados (DataHora)	normal
idx_acoes_realizadas_datahora	acoes_realizadas (DH_ACAO)	normal
idx_modo_atual_torre_usuario	modo_atual_torre (ID_USER)	normal
idx_objetos_detectados_perfil	objetos_detectados (PerfilAlvoId)	normal
idx_ocorrencia_estados_objeto	ocorrencia_estados (ObjetoDetectadoId)	normal
idx_objetos_detectados_rastreamento	objetos_detectados (RastreamentoId, Quadrante)	único, parcial
idx_cameras_dispositivo	cameras (DispositivoId)	único
idx_cameras_quadrante	cameras (Quadrante)	único

O índice único parcial `idx_objetos_detectados_rastreamento` (`WHERE RastreamentoId IS NOT NULL`) é o que impede duplicar a ocorrência do mesmo acompanhamento num mesmo quadrante, inclusive quando várias câmeras processam em paralelo — sem restringir o radar, que não gera `RastreamentoId` e pode repetir quadrante legitimamente.

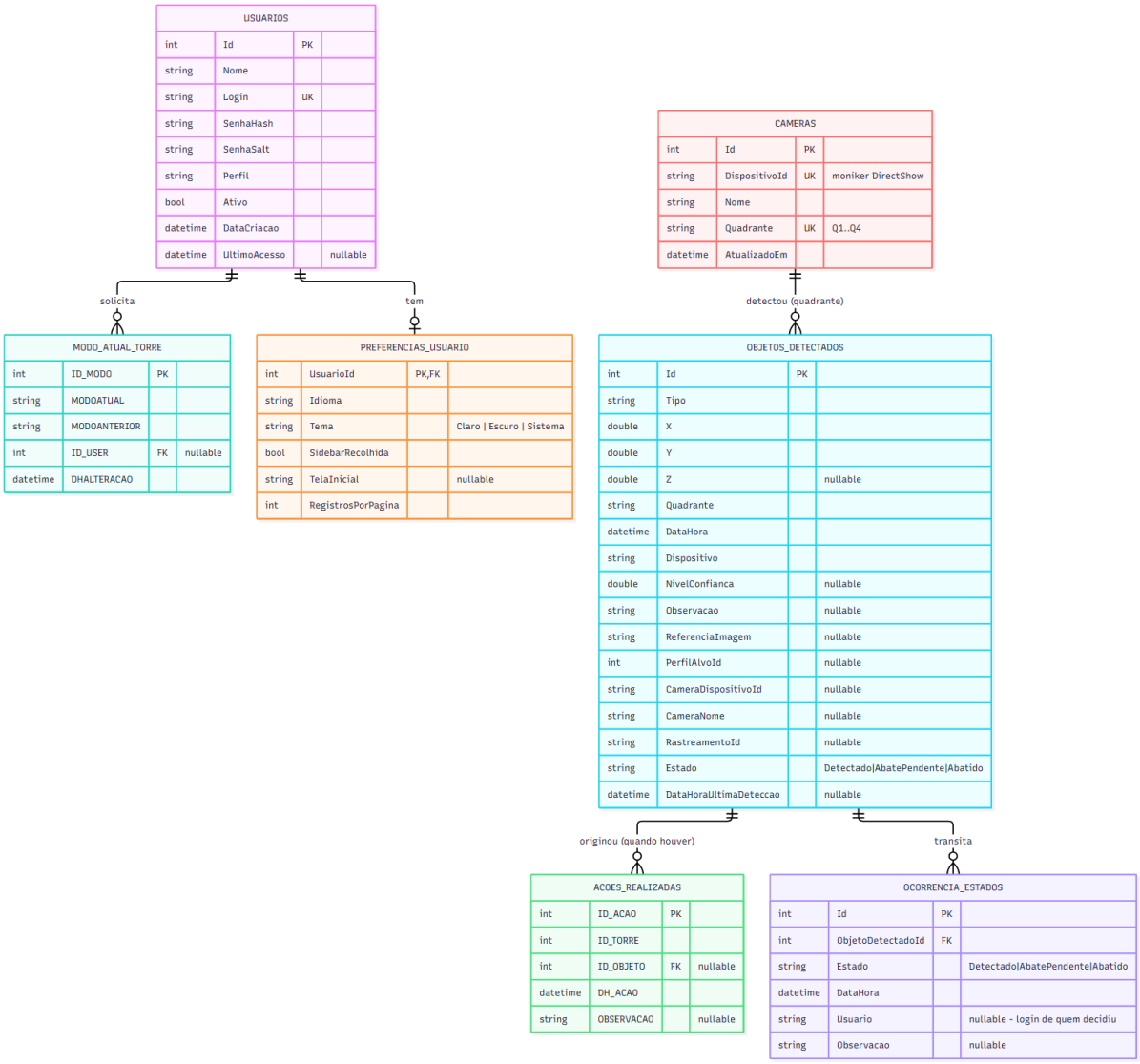


Figura 1: Modelo de dados atual (SQLite) — diagrama entidade-relacionamento levantado a partir do schema em produção (`schema.sql`).

6 Conclusão

A persistência atual, baseada em SQLite, atende ao estágio de protótipo do RadarTorres com integridade referencial real, auditoria consistente e instalação/execução totalmente locais. A escolha do SGBD — detalhada na Seção 2.2 — equilibra segurança (menor superfície de ataque, ausência de servidor exposto) e leveza (sem instalação ou administração de infraestrutura de banco), dentro do perfil de uso de nó único do sistema. O uso de uma interface de repositório dedicada para cada entidade mantém a camada de dados desacoplada das demais camadas da aplicação, viabilizando uma eventual migração futura para um SGBD cliente-servidor sem exigir mudanças em ViewModels ou Services.

REFERÊNCIAS

[REFERÊNCIA BIBLIOGRÁFICA A INSERIR]